Project Summary

The goal of this project is to build an artificial intelligence model that can automatically detect and pinpoint electrical faults (like short circuits or line failures) within a smart grid. Specifically, the project models the "IEEE 39-Bus" system, which is a standard representation of a power grid. By doing this, the system can quickly figure out exactly *where* a problem is happening and *what kind* of problem it is to help keep the power grid reliable.

What Was Done (The Workflow)

The project follows a complete 7-phase pipeline:

- **Phase 1 (Grid Setup):** The physical power grid was converted into a mathematical "graph" where power stations (buses) are the nodes, and the transmission lines connecting them are the edges.
- **Phase 2 (Data Generation):** Because real-world grid failures are rare, the team used a power system simulator to generate synthetic (fake) fault data. They simulated different types of electrical faults under various conditions.
- **Phase 3 (Data Processing):** The data was broken down into "sliding windows" (small chunks of time) to help the model learn how power signals fluctuate right before and during a fault.
- **Phase 4 (Model Building):** The team built the core AI model: a Spatio-Temporal Graph Neural Network.
- **Phase 5 (Training):** The model was trained using a "multi-task" approach, meaning it was taught to simultaneously predict the fault's location and classify the fault's type.
- **Phases 6 & 7 (Evaluation & Testing):** Finally, the model's accuracy was evaluated, and it was put through "robustness" tests to see how well it performs when the data is noisy or if a transmission line is unexpectedly offline.

Implementations

The core implementation in this project is the **ST-GNN (Spatio-Temporal Graph Neural Network)** model. It combines two powerful machine learning concepts to understand the grid:

- **GCN (Graph Convolutional Network):** This handles the "Spatial" (physical) part. It looks at the map of the grid to understand how the different power buses are physically wired to one another.
- **LSTM (Long Short-Term Memory):** This handles the "Temporal" (time) part. It acts like a timeline reviewer, tracking how power signals (like voltage and frequency) change second-by-second to spot abnormal patterns.

Technologies Used

The pipeline relies on several specialized Python libraries:

- **PyTorch & PyTorch Geometric:** The core AI frameworks used to build, train, and run the neural network on a graphics card (GPU).
- **Pandapower:** A simulation tool used to digitally recreate the electrical grid and generate the synthetic fault data.
- **Scikit-learn & Imbalanced-learn:** Used for preprocessing the data, calculating accuracy scores, and balancing the dataset (ensuring the AI sees enough examples of all fault types).
- **Pandas, Numpy, Matplotlib, Seaborn:** Standard data science tools for handling numbers, organizing data tables, and drawing visual charts.

Presentation:

Speaker 1: Anik Kumar Basu (The Hook & The Vision)

Anik: "Hello everyone. My name is Anik, and along with my teammates Sabrina and Adive, we are here to talk about our project: Spatiotemporal GNNs for Real-Time Fault Localization in Power Grids.

Imagine a sudden fault in a power grid. Fast fault detection and exact bus-level localization are absolutely essential for keeping modern power grids resilient, especially as we integrate more renewable energy. But there is a major problem with how we currently do this.

Many conventional relays struggle to capture how these sudden disturbances actually propagate through the network. On the other hand, standard machine learning models, like CNNs, aren't naturally designed to understand the physical layout—or the topology—of the power grid.

Our solution is a Spatiotemporal Graph Neural Network, or ST-GNN. It looks at *both* the 'where'—using spatial message passing on the grid graph—and the 'when'—using temporal sequence modeling. To explain exactly how we built this and taught it to understand the grid, I'll hand it over to Sabrina."

Speaker 2: Sadik Mahmud Adive (The Inner Workings & Training)

Adive: "Thanks, Anik. Because real-world fault event logs from power utilities are highly confidential, we first had to build a massive simulation.

We used a tool called Pandapower to generate around 60,000 synthetic samples on the IEEE 39-bus New England system. To make sure the model learned real-world conditions, we simulated four different fault types, four fault resistances, and three different load levels.

So, how does the model process this data? First, the data passes through two Graph Convolutional Network, or GCN, layers. This acts as the 'spatial brain.' It passes neural messages along the edges of the graph, allowing each node to gather electrical state information from its neighbors.

Next, the data flows into a two-layer Long Short-Term Memory, or LSTM, module. This is the 'temporal brain.' It watches sliding windows of PMU data—representing 500 milliseconds of time—to analyze how the transient fault develops.

We trained this entire setup with a multi-task objective: its primary job is to pinpoint the exact fault bus, while it simultaneously works to classify the exact type of electrical fault. Now, Adive will share how well this complex model actually performed."

Speaker 3: Sabrina Sonda Snaha (The Results & Conclusion)

Sabrina: "Thank you, Adive. I'm excited to share that our ST-GNN model performed exceptionally well.

When put to the test, it achieved a 92.8% top-1 localization accuracy and a 97.4% fault detection accuracy. It comfortably outperformed classical thresholding baselines, as well as standard ML methods like Random Forests and MLPs.

But the real world is messy, so we also tested its robustness. We found that even if we add typical PMU measurement noise, the model maintains over 90% accuracy. We also tested N-1 contingencies—meaning we unexpectedly removed a transmission line from the network. Impressively, the localization accuracy dropped by an average of only 4.2 percentage points, proving the model can adapt to changes in the grid's topology.

Through ablation studies, we also proved that both the spatial GCN and temporal LSTM components are absolutely necessary; removing either one causes performance to drop.

Best of all, our model is highly efficient. It has only about 1.39 million parameters and boasts an inference latency of just 6.2 milliseconds. This makes it lightweight enough to be readily deployed on embedded edge hardware in the real world.

Thank you for your time, and we would now be happy to answer any questions."